

# CASE STUDY REPORT

## Comparative Case Study on Library Management System

**Submitted by:** A. Hemanth and B. Shashi Nayak

**Roll Numbers:** 160124748094, 160124748098

**Date of Submission:** /09/2026

**Year/Sem/Sec:** III Year / V Semester / CSM-2

**Submitted to:** Smt. G. Baby Lakshmi Prasanna

**Department:** CSE (AI & ML)

# Case Study : Library Management System

---

## 1. Introduction

### 1.1 Problem Statement

Traditional library management often involves maintaining book and student records manually. Searching for books, checking availability, registering students, issuing books, returning books, and calculating fines can become time-consuming and may lead to errors.

To overcome these limitations, this case study presents a web-based Library Management System that provides a simple and interactive interface for managing common library operations.

The system allows users to browse books, search and filter books, register students, issue books, return books, and calculate fines.

### 1.2 Importance in Full Stack Development

Library Management System is a suitable Full Stack Development case study because it demonstrates how different web technologies work together to create a dynamic application.

The project demonstrates:

- HTML5 for web page structure
- CSS3 for styling
- Bootstrap 5 for responsive design
- JavaScript for application logic and validation
- ReactJS for component-based UI development
- React Router for navigation
- Fetch API for asynchronous data retrieval
- JSON for structured data representation
- LocalStorage for persistent browser-side storage

The project also demonstrates important React concepts such as JSX, components, props, events, state, useState and useEffect.

### 1.3 Real-World Applications

The same concepts can be extended to real-world systems such as:

- College and university libraries
- Public libraries
- Digital book management systems
- School library systems
- Institutional resource management systems
- Online book rental systems

---

## 2. Methods Compared

### 2.1 Traditional Library Management

Traditional library management relies mainly on manual records and physical registers.

Typical operations include:

- Maintaining book records manually
- Recording student details
- Searching through records manually
- Recording book issue and return details
- Calculating fines manually
- Updating availability records manually

Although this approach is simple, it becomes inefficient when the number of books and students increases.

## 2.2 Proposed React-Based Library Management System

The proposed system uses modern web development technologies to digitize common library operations.

The system provides:

- Dynamic book listing
- Book search and filtering
- Student registration
- Registered student dashboard
- Automatic student name retrieval
- Book issue functionality
- Book return functionality
- Fine calculation
- Responsive user interface
- Persistent student information using LocalStorage

## 2.3 Comparison

| Feature              | Traditional System                           | Proposed Web System                                      |
|----------------------|----------------------------------------------|----------------------------------------------------------|
| Book Search          | Manual searching through a physical catalog  | Available/Issued filters on a digital screen, Search box |
| Student Registration | Register with handwriting                    | Web form with fields                                     |
| Student Records      | Physical cabinets and papers                 | Digital dashboard with graphs and data                   |
| Book Issue           | Pen and paper manual entry                   | Online form with submit button                           |
| Book Return          | Calculating fines on paper with a calculator | Automated fine calculation displayed digitally           |
| Data Loading         | Person carrying heavy stacks of paper        | Fetch API + JSON                                         |
| UI Updates           | Manual re-drawing or constant refreshing     | React dynamic updates                                    |
| Data Persistence     | Dusty physical archives                      | LocalStorage with browser                                |
| Navigation           | Physical registers spread out                | React Router with a menu and different pages on a screen |

## 2.4 Technologies Used

- **ReactJS:** Used to create reusable and dynamic user interface components.
- **JavaScript:** Used for application logic, events, searching, filtering and validation.
- **Bootstrap 5:** Used for responsive layouts, buttons, forms, tables and UI styling.
- **Fetch API:** Used to asynchronously retrieve JSON data without reloading the complete page.
- **JSON:** Used to store and represent initial book and student data.
- **React Router:** Used to navigate between different pages of the application.
- **Local Storage:** Used to persist registered student information in the browser.

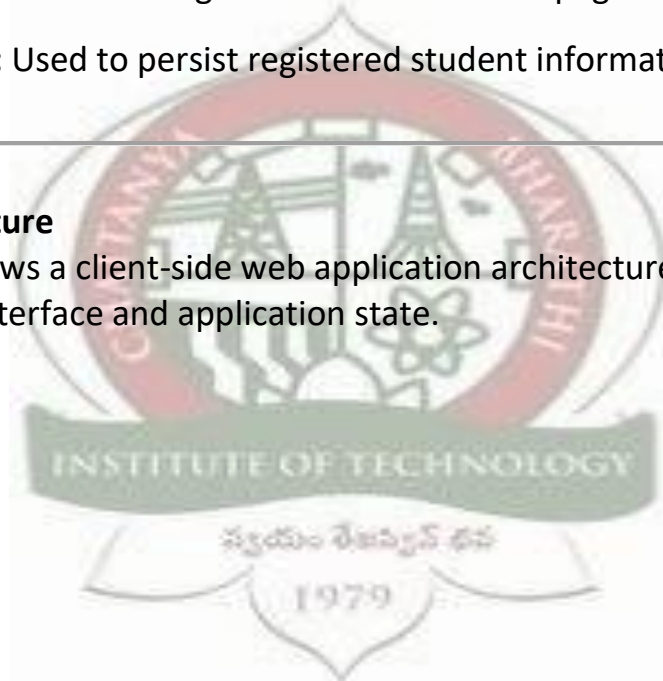
---

## 3. Architecture

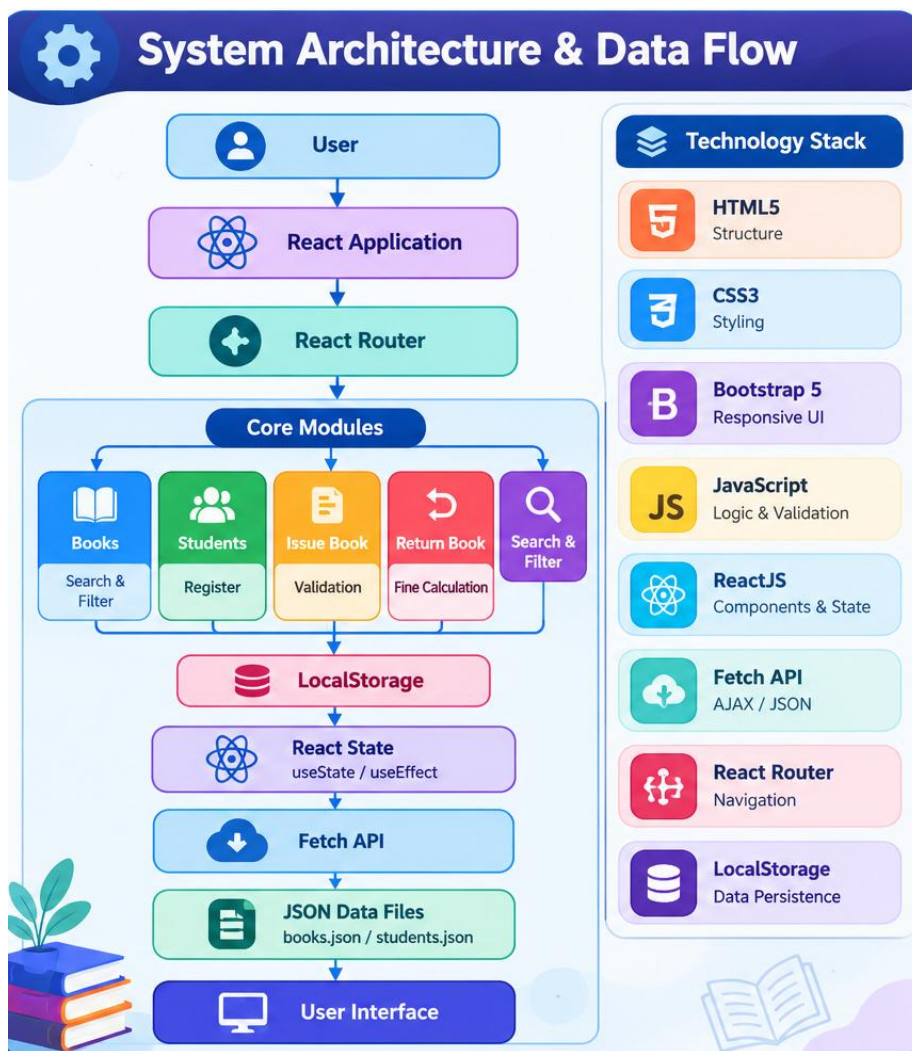
### 3.1 Logical Architecture

The application follows a client-side web application architecture where React manages the user interface and application state.

**Logical Flow:**



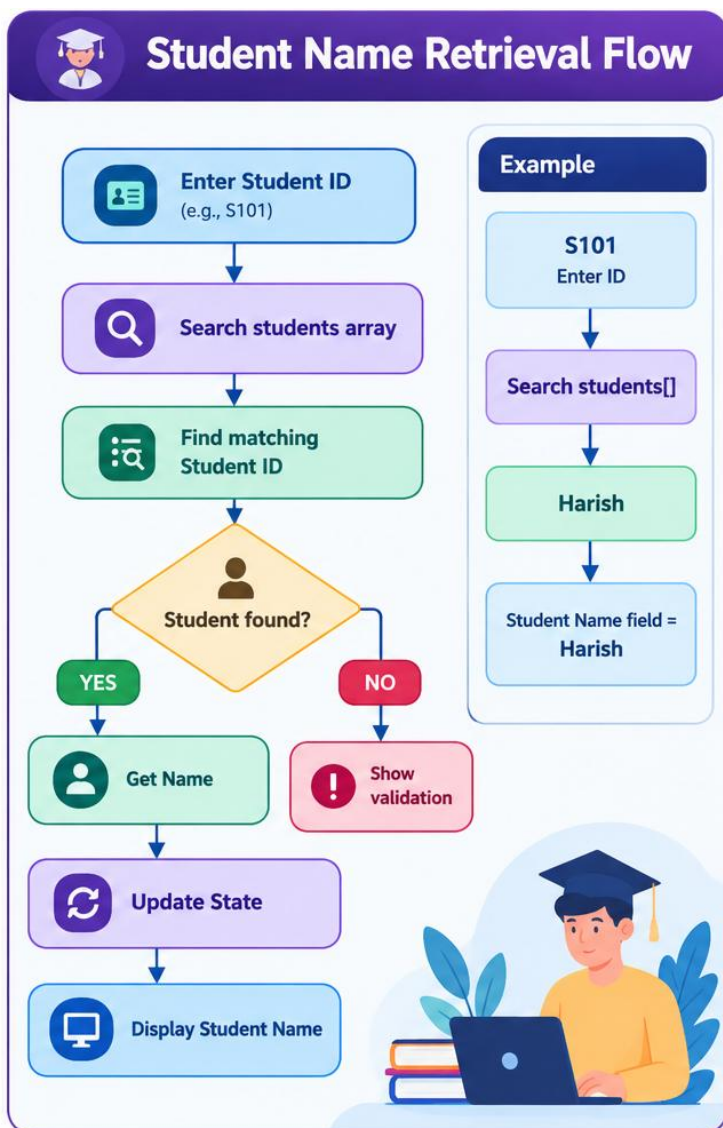




### 3.2 Request and Data Flow

1. The user opens a page using the navigation bar.
2. React Router displays the required page/component.
3. The React component loads the required data.
4. Fetch API retrieves the JSON data asynchronously.
5. The received response is converted into JavaScript objects.
6. The data is stored in React state using useState.
7. React dynamically displays the data in the user interface.
8. User actions such as search, filtering, registration, issue and return update the application state.
9. Registered student information is stored in LocalStorage for persistence.

### 3.3 Student Name Retrieval Flow



### 3.4 Book Data Flow

books.json



Fetch API



response.json()



useState()



BookList



BookCard



Book displayed on screen

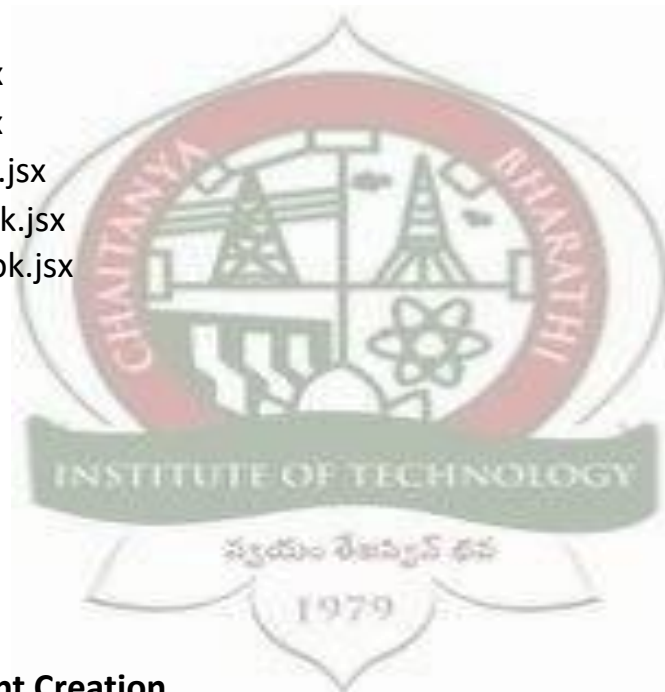
## 4. Implementation

### 4.1 Project Structure

The React project is organized into pages and components.

my-react-app/

```
|
|├── public/
| |├── books.json
| |└── students.json
|
|├── src/
| |├── components/
| | |└── BookCard.jsx
| |
| |├── pages/
| | |├── Home.jsx
| | |├── Books.jsx
| | |├── Students.jsx
| | |├── IssueBook.jsx
| | |└── ReturnBook.jsx
| |
| |├── App.jsx
| |├── App.css
| |├── index.css
| |└── main.jsx
|
|├── package.json
|└── vite.config.js
```



## 4.2 React Component Creation

A reusable BookCard component is used to display individual books.

```
function BookCard({ book }) {
  return (
    <div className="book-card">
      <h3>{book.title}</h3>
      <p>ID: {book.id}</p>
      <p>Author: {book.author}</p>
      <p>Category: {book.category}</p>
      <p>Status: {book.status}</p>
    </div>
  );
}
```

export default BookCard;

The component receives book information through **props** and dynamically displays it.

## 4.3 Fetch API and JSON

Book information is loaded from the JSON file using the Fetch API.

```
useEffect(() => {  
  fetch("/books.json")  
    .then(response => response.json())  
    .then(data => setBooks(data));  
}, []);
```

Here:

- `fetch()` requests the JSON file.
- `response.json()` converts the response into JavaScript data.
- `setBooks()` stores the data in React state.
- `useEffect()` performs the data-loading operation when the component is loaded.

#### 4.4 Student Data Loading

Student records are loaded using the same Fetch API approach.

```
useEffect(() => {  
  fetch("/students.json")  
    .then(response => response.json())  
    .then(data => setStudents(data));  
}, []);
```

#### 4.5 Student Name Retrieval

When a Student ID is entered, the application searches the student array.

```
const student = students.find(  
  s => s.id.toLowerCase() === studentId.toLowerCase()  
);
```

```
if (student) {  
  setStudentName(student.name);  
}
```

This allows the student's name to appear automatically after entering a valid Student ID.

#### 4.6 Book Search

The search functionality filters books according to the entered title or author.

```
const filteredBooks = books.filter(book =>  
  book.title.toLowerCase().includes(search.toLowerCase()) ||  
  book.author.toLowerCase().includes(search.toLowerCase())  
);
```

This provides real-time client-side searching without reloading the page.

#### 4.7 Book Availability Filtering

Books are filtered according to their status.

```
const filteredBooks = books.filter(book => {  
  if (filter === "Available")  
    return book.status === "Available";
```



```
if (filter === "Issued")  
    return book.status === "Issued";
```

```
    return true;  
});
```

#### 4.8 LocalStorage

Registered student information is stored in LocalStorage.

```
localStorage.setItem(  
    "students",  
    JSON.stringify(students)  
);
```

The saved data can later be retrieved using:

```
const savedStudents =  
    JSON.parse(localStorage.getItem("students")) || [];
```

LocalStorage allows student records to remain available after refreshing the browser.

#### 4.9 Fine Calculation

The return-book module calculates the fine based on the number of days the book was kept.

```
const fine = days > 7 ? (days - 7) * 5 : 0;
```

The first seven days are considered within the allowed period. If the book is kept longer, a fine is calculated for the additional days.

#### 4.10 React Router

React Router is used to navigate between different sections.

```
<Routes>  
  <Route path="/" element={<Home />} />  
  <Route path="/books" element={<Books />} />  
  <Route path="/students" element={<Students />} />  
  <Route path="/issue" element={<IssueBook />} />  
  <Route path="/return" element={<ReturnBook />} />  
</Routes>
```

This provides navigation between the Home, Books, Students, Issue Book and Return Book pages.

---

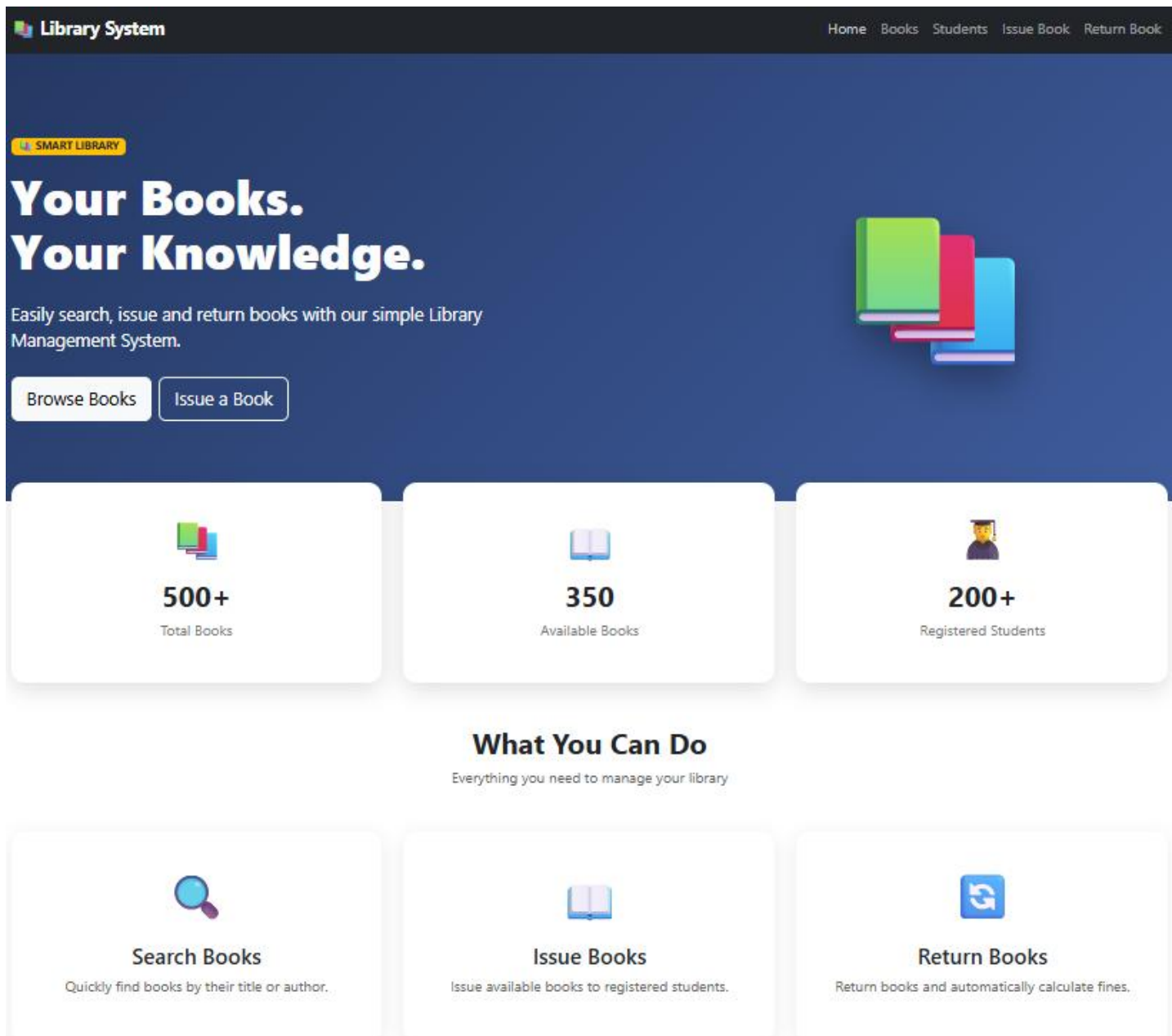
### 5. Results

The developed Library Management System successfully provides an interactive interface for common library operations.

#### 5.1 Home Page

The Home page provides an overview of the system and navigation to major library operations.

#### Output Snapshot: Home Page



## 5.2 Books Page

The Books page displays books dynamically with their ID, title, author, category and availability status.

The page also provides:

- Search by title or author
- All books filter
- Available books filter
- Issued books filter
- Book details

**Output Snapshot: Books Page**

 Library Books Search by title or author...

All

Available

Issued



## Java Programming

ID: 101

Author: James Gosling

Category: Programming

Status: Available

View Details



## Database Management Systems

ID: 102

Author: Korth

Category: Database

Status: Issued

View Details



## Operating Systems

ID: 103

Author: Galvin

Category: Systems

Status: Available

View Details



## Python Programming



## Computer Networks



## Artificial Intelligence

### 5.3 Registered Students Dashboard

The Students page allows new students to be registered using Student ID, Student Name, Branch and Year.

The registered students are displayed in a structured table.

**Output Snapshot: Registered Students**



## Registered Students

### + Add New Student

Student ID

S105

Student Name

Enter name

Branch

Select Branch



Year

Select Year

Add Student

### Student Records

| Student ID | Student Name | Branch | Year |
|------------|--------------|--------|------|
| S101       | Harish       | CSE    | 3    |
| S102       | Rajesh       | AIML   | 3    |
| S121       | Dhanush      | ECE    | 2    |
| S104       | Tanish       | CSE    | 4    |
| S115       | Ananya       | AIML   | 2    |
| S118       | Sneha        | CSE    | 3    |

## 5.4 Issue Book

The Issue Book page accepts Student ID and Book ID.

After entering a valid Student ID, the corresponding student name is automatically displayed.

### Output Snapshot: Issue Book



## Issue Book

Student ID

Example: S101

Student Name

Student name will appear automatically

Book ID

Example: 101

Issue Book

## 5.5 Return Book

The Return Book page accepts the number of days the book was kept and calculates the applicable fine.

### Output Snapshot: Return Book

 **Return Book**

Number of Days Book Was Kept

Example: 10

Calculate Fine

---

## 5.6 Result Summary

The implemented system successfully demonstrates:

- Responsive library dashboard
- Dynamic book display
- Book search and filtering
- Student registration
- Registered student dashboard
- Student ID validation
- Automatic student name retrieval
- Book issue functionality
- Book return functionality
- Fine calculation
- Fetch API and JSON data handling
- React component-based development
- React state management
- Client-side data persistence

---

## 6. Conclusion

The Library Management System successfully demonstrates the application of modern web development concepts in a practical case study.

The system combines HTML5, CSS3, Bootstrap 5, JavaScript and ReactJS to create a responsive and interactive interface. React components provide modularity and reusability, while useState and useEffect help manage dynamic application data.

The Fetch API is used for asynchronous retrieval of JSON data, demonstrating the AJAX concept. React Router provides navigation between different modules, while LocalStorage provides browser-side persistence for registered student information. The project demonstrates how a traditional manual library process can be converted into an interactive digital system with improved searching, record management and



transaction handling.

### Future Scope

The current system can be further enhanced by:

- Developing a Node.js and Express backend
- Connecting the application to MySQL or MongoDB
- Implementing user authentication and authorization
- Adding librarian and student login roles
- Maintaining complete issue/return transaction history
- Adding real-time database synchronization
- Sending email or notification reminders for overdue books
- Adding book reservation functionality
- Deploying the application on a cloud platform
- Providing an administrative analytics dashboard

A complete production version could therefore evolve from the current React-based prototype into a full-stack application with a dedicated backend, database and authentication system.

---

### 7. Presentation:

